SafeTrail — Technical Reference

Data Structures course project · github.com/adivishall/safetrail

Generated from docs/final/ by tools/build_pdfs.py

Complete technical documentation

Everything needed to build, run, test, operate and extend SafeTrail. For the design see ARCHITECTURE.md; for the structures see DATA_STRUCTURES_EXPLAINED.md.

1. Requirements

Need	Version used	Notes
C++17 compiler	Apple clang 17 (macOS), g++ 13 and clang on Linux (CI)	nothing else to install on a Mac with the Command Line Tools
make	macOS `/usr/bin/make` 3.81 or GNU make	
Python 3	3.13 (any 3.8+)	tests' end-to-end check, benchmark rendering, mutation check — standard library only
A browser	Chrome, Safari, Firefox	for the operations console
Optional	CMake ≥ 3.16	`make cmake-build` builds the same sources

On a MacBook Air M4: `xcode-select --install` once, then everything below. No database, map service, API key or network connection is needed.

2. Install, build, run

```
git clone https://github.com/adivishall/safetrail.git
```

```
cd safetrail && make app
```

`make app` compiles the library, the server and its dependencies (about a minute the first time) and starts the server on **http://127.0.0.1:8080**. The first start prints the four accounts (`admin`, `operator`, `responder`, `tourist`) and their generated passwords, and saves them in `var/demo-credentials.txt` (mode 0600, git-ignored). The walkthrough scenario is loaded and paused at T+00:00:00.

Mode	Command	What differs
Development / demo (default)	`make app`	demo responders (labelled DEMO), scenarios enabled, clock driven by the scenario runner
Production	`make app APP_ARGS="--mode production"`	no fixtures, scenarios refused, real-time clock, only the `admin` account
Another port	`make app APP_ARGS="--port 9000"`	
Choose the preloaded scenario	`make app APP_ARGS="--scenario entry"`	`walkthrough`, `normal`, `entry`, `separation`, `offline`, `none`
Fixed passwords	`SAFETRAIL_ADMIN_PASSWORD=... make app`	likewise `_OPERATOR_`, `_RESPONDER_`, `_TOURIST_`; at least 10 characters
Reset	`make app-reset`	deletes `var/`; passwords regenerate on the next start

Demo data: there is nothing to initialise. The world is built at start-up from `data/zones/shillong_osm.geojson` (38 OSM hazard zones) and `data/osm/roads.txt` (144,090 junctions) in about 0.4 s. Reloading a scenario (Live Monitoring → Scenario → Load, or **Reset** in the top bar) rebuilds the world from scratch.

3. Every make target

Target	Does
`make`	build the three programs
`make app` / `make app-reset`	run the platform / delete local credentials
`make test`	51 test files, 12,066 checks (about 20 s once built)

Target	Does
`make e2e`	start the server on a free port and drive it over HTTP (29 checks)
`make bench`	every benchmark (≈ 5 min); rewrites `bench/results/*.csv` and the tables in README and RESULTS
`make validate`	docs links, results tables, `-Werror`, tests, determinism, e2e, sanitizers, CMake + ctest
`make mutation`	inject 28 known bugs; each must be caught
`make sanitize`	UBSan (macOS) or ASan + UBSan (Linux) over the whole suite
`make demo` / `make dashboard` / `make review`	the original CLI demo, the static dashboard, the scripted review walkthrough

4. Configuration (`app::PlatformConfig`)

Setting	Default	Editable at run time
trail retention	6 h	yes (Settings)
breach → incident at severity ≥	3	yes
warnings become alerts at severity ≥	4	yes
approach alert if ETA ≤	180 s	yes
GPS jump threshold	70 m/s	yes
auto-assign recommended responder	off	yes
escalation timeout	120 s	start-up
cohesion pass interval	10 s	start-up
hysteresis (enter / exit margin, samples, min dwell)	15 m / 25 m / 3 / 5 s	start-up
correlation radius / window	200 m / 10 min	start-up
PBKDF2 iterations	120,000	start-up
limits	5,000 users, 20,000 zones, 4,096 vertices per zone	start-up

5. HTTP API

All responses are JSON unless noted. Mutating requests need the header `X-SafeTrail-Request: 1` and the session cookie set by `/api/login`. Roles: **A** admin, **O** operator, **R** responder, **T** tourist; "self" means the user named in the path.

Method and path	Who	Purpose
`GET /api/health`	anyone	mode, clock
`POST /api/login {username, password}`	anyone	sets the session cookie
`GET /api/session`	anyone	current user or null
`POST /api/logout`	signed in	ends the session
`GET /api/live`	signed in (filtered by role)	users, responders, incidents, groups, runner, counts
`GET /api/users`	A O (all), R (assigned subjects), T (self)	accounts
`POST /api/users {username, display_name, role, password}`	A	register
`GET /api/users/{id}`	A O, R if assigned, self	profile, location, zones, battery model (audited)
`PATCH /api/users/{id} {active?, display_name?, password?}`	A; self for name/password	

Method and path	Who	Purpose
`GET /api/users/{id}/trail?from=&to=`	A O, R if assigned, self	movement history (audited)
`POST /api/users/{id}/consent {consent}`	self	give / withdraw
`POST /api/users/{id}/sharing {active}`	self	start / stop the session
`POST /api/users/{id}/fix {lat, lon, accuracy_m, t_ms?, seq?}`	self	ingest a GPS fix
`POST /api/users/{id}/sos`	self	severity-5 SOS
`POST /api/users/{id}/battery {aware}`	self, A	adaptive sampling
`DELETE /api/users/{id}/history`	self, A	erase movement history
`GET /api/groups` · `POST /api/groups {name}`	A O	
`POST /api/groups/{id}/members {user, action}` · `PATCH /api/groups/{id} {proximity_m, straggler_m, confirm_s}`	A O	
`GET /api/zones` · `GET /api/zones/{id}`	signed in	geometry; detail adds version history, change log, incidents
`POST /api/zones` · `PATCH /api/zones/{id} {name, kind, severity, ring, max_dwell_s, active_from_ms, active_to_ms}`	A	validated polygon
`DELETE /api/zones/{id}` · `POST /api/zones/{id}/validity {active}` or `{from_ms, to_ms}`	A	
`POST /api/zones/import` (GeoJSON body)	A	all-or-nothing import
`GET /api/alerts?state=open`	A O (all), T (own)	open alerts in triage order
`POST /api/alerts/{id}/ack\`	resolve\	dismiss`	A O	
`GET /api/incidents` · `GET /api/incidents/{id}`	A O, assigned R, affected T	detail adds alerts and evidence timeline
`POST /api/incidents/{id}/ack`	A O, assigned R	
`POST /api/incidents/{id}/assign {responder?}`	A O	default = recommended; A* route
`POST /api/incidents/{id}/resolve {note}`	A O, assigned R	
`POST /api/dispatch/plan {apply}`	A O	greedy vs Hungarian over unassigned incidents
`GET /api/responders` · `POST /api/responders {name, type, lat, lon}` · `PATCH /api/responders/{id} {available?, lat?, lon?}`	read: signed in; create: A; update: A O / own unit	
`GET /api/roads? min_lat=&min_lon=&max_lat=&max_lon=`	signed in	road segments in a box (map base layer)
`GET /api/history?t=`	A O	what the system knew at t (audited)
`GET /api/events? limit=&user=&kind=&order=lamport`	A O (all), T (own)	event log
`GET /api/evidence` · `GET /api/evidence/{i}`	A O	Merkle root; one entry with its inclusion proof

Method and path	Who	Purpose
`GET /api/audit`	A	location-access audit log
`GET /api/stats`	A O	live counters
`GET /api/settings` · `PATCH /api/settings`	A O read; A write	
`GET /api/scenario` · `POST /api/scenario/load\`	start\	pause\	resume\	reset\	step\	speed`	A O, demo mode	
`GET /api/lab` · `POST /api/lab/{experiment}`	signed in	spatial, kdtree, zorder, avl, persistent, graph, heap, assignment, dsu, hash, wheel, ring, merkle, lamport, geometry
`GET /api/bench` · `GET /api/bench/{file}`	signed in	committed benchmark CSVs (text)

Errors: `{"error": "..."}` with 400 (bad input), 401 (no session), 403 (role or CSRF), 404, 409 (state conflict, e.g. responder busy), 411/413/431/501 (HTTP limits), 429 (login throttled).

6. Data formats

- **Zones:** GeoJSON FeatureCollection, `[lon, lat]` order, closed rings; properties `name`, `kind` (`restricted` / `caution` / `advisory` / `safe`), `severity` 1–5, `max_dwell_s`, `active_from_s`, `active_to_s`, `synthetic`, `jurisdiction`, `enter_margin_m`, `exit_margin_m`.
- **Roads:** `safetrail-roads 1|2`, node count, `lat lon` lines, edge count, `u v weight_m` lines (v2 directed).
- **Evidence entry (canonical form hashed into the Merkle log):** `safetrail-log-v1|seq|lamport|device|t_ms|recorded_ms|kind|user|actor|zone|alert|incident|group|responder|late|live|detail`.
- **Platform clock:** milliseconds since the platform started (demo: simulated).

7. Extending

- A new index: implement `index::SpatialIndex`, add it to `make_index`, and to `tests/index/differential_test.cpp` and `tests/fence/index_independence_test.cpp`.
- A new alert rule: raise it in `Platform` with `raise_alert`; decide in `incident_worthy` whether it opens incidents.
- A new Lab experiment: a function in `src/app/lab.cpp` returning the structure's real state, and a view in `web/app/js/pages/lab.js`.
- A new scenario: a builder in `src/app/scenario.cpp` plus a test in `tests/app/scenario_test.cpp` asserting the story it tells.

8. Troubleshooting

Symptom	Cause / fix
`cannot listen: Address already in use`	another server on 8080: `make app APP_ARGS="--port 8090"`
Forgot the passwords	`cat var/demo-credentials.txt`, or `make app-reset` and restart
Login says "too many failed attempts"	wait 60 s
Map shows no roads	zoom in: roads are drawn below about 18 m per pixel
"Scenarios are disabled"	the server was started in production mode
A page says the server is not reachable	the server stopped; restart `make app` (state is in memory)

Architecture

SafeTrail has three layers: the **engine** (data structures, geometry, geofencing — present before the final phase), the **platform** (users, consent, sessions, alerts, incidents, groups, responders, history, offline sync), and the **server and web app** (HTTP API, scenario runner, operations console, Data Structures Lab). The static dashboard (`make dashboard`, deployed to GitHub Pages and Vercel) is a fourth, independent product: a replay of one simulated run. The engine's pipeline and design decisions are described in detail in ../ARCHITECTURE.md.

1. Layers

(Diagram: rendered on GitHub; see the figures in this document.)

2. One GPS fix, end to end

(Diagram: rendered on GitHub; see the figures in this document.)

3. Life cycles

(Diagram: rendered on GitHub; see the figures in this document.)

(Diagram: rendered on GitHub; see the figures in this document.)

4. Offline phone

(Diagram: rendered on GitHub; see the figures in this document.)

5. Concurrency and ordering

The server is single-threaded on purpose. `net::Server::run` multiplexes all sockets with `poll(2)`, calls the API handler for each complete request, and between wake-ups advances the simulation clock. Every mutation of the platform therefore happens in one total order, the order of the event log, with no lock anywhere. Per user, fixes must arrive in time order to be evaluated (the hysteresis filter is a state machine over time); late fixes are recorded but not replayed. Measured cost: about 3–4 µs per fix through the whole ingestion path, flat from 10 to 5,000 users (benchmark section 19).

6. Time

The platform clock is milliseconds since the platform's own start. In demo mode the scenario runner advances it (pause really stops time; 0.25×–60× speeds); in production mode it follows the wall clock. Every log entry and trail point records both the time an event happened and the time the server learned of it, which is what lets the Investigation page answer "what did the system know at t?".

7. Repository map

Path	Contents
`include/safetrail/`, `src/`	engine (`geo`, `index`, `ds`, `fence`, `track`, `alert`, `group`, `graph`, `dispatch`, `sync`, `evidence`, `power`, `jurisdiction`, `sim`, `viz`, `util`), platform (`app`), HTTP (`net`)
`apps/`	`safetrail_server`, `safetrail_headless` (CLI + static dashboard), `safetrail_bench`
`web/app/`	the operations console and Lab (HTML, CSS, ES modules)
`tests/`	51 test files: unit, property, differential, golden, platform, API, HTTP
`tools/`	benchmark rendering, mutation check, end-to-end HTTP check, OSM converters
`data/`	OSM hazard zones and road network around Shillong
`bench/results/`	committed benchmark CSVs (the evidence the reports quote)

Path	Contents
`docs/final/`	the final academic documentation (this folder)

Data structures, explained

One section per structure, each with the same parts: the problem, why this structure, how it works, how this repository implements it, where SafeTrail uses it, cost, and what it is compared against. Each can be watched running on the real code in the web app's **Data Structures Lab**.

The deeper invariants and proofs are in ../DATA_STRUCTURES.md; measured costs in ../RESULTS.md.

---

1. Brute force — the baseline and the oracle

- **Problem:** which hazard zones' boxes overlap the box around a GPS fix?
- **How:** test every box. Θ(n) per query.
- **Implementation:** `index::BruteForceIndex` (`src/index/brute_force.cpp`), a vector scan.
- **Why keep it:** it is obviously correct, so every fast index is compared against it — in the tests (`tests/index/differential_test.cpp`), in the benchmark, and live in the Lab, which prints the mismatch count for every query.

2. Quadtree — partition space

- **Problem:** the same query, without looking at every zone.
- **How:** a square region split into four children once it holds more than `capacity` boxes (default 8, max depth 12). A box that straddles a split line stays in the internal node. A query descends only into children whose region meets the query box.
- **Implementation:** `index::Quadtree` (`src/index/quadtree.cpp`). The root is fitted to the data, not to the whole planet (a world root wastes ~11 levels on Meghalaya); inserting outside the root doubles it outward; deleting collapses a subtree that has fallen below capacity.
- **Cost:** O(log n + k) expected, O(n) worst case (all boxes straddling the centre).
- **Lab:** shows every node and, step by step, which nodes the query entered (teal) and pruned (red).

3. R-tree — partition items

- **Problem:** the same, with nodes that adapt to where the zones actually are.
- **How:** every node is the minimum bounding rectangle (MBR) of its children; siblings may overlap. All leaves are at the same depth. On overflow, Guttman's **quadratic split** picks the two entries that would waste most area together as seeds and distributes the rest.
- **STR bulk loading:** sort all boxes by centre longitude, cut into ⌈√P⌉ vertical slices (P = leaf count), sort each slice by latitude, pack runs of `M`. Fewer, fuller, less-overlapping nodes than n inserts.
- **Implementation:** `index::RTree` (`src/index/rtree.cpp`); `build()` is STR, `build_incremental()` keeps the insert-built tree for comparison; removal condenses under-full nodes and reinserts orphans.
- **Where:** the platform's live index (`app::Platform`).
- **Cost:** O(log n + k) expected, O(n) worst; STR build O(n log n).

4. Geohash / Z-order — a flat, shippable index

- **Problem:** an index a phone can download as bytes and use with no network.
- **How:** interleave the bits of latitude and longitude (Morton code) of each box centre into a 48-bit key; sort; answer range queries by padded key ranges and binary search.
- **Implementation:** `index::Geohash` (`src/index/geohash.cpp`) with `serialize()/deserialize()`.
- **Where:** an offline phone in the platform evaluates zones from a geohash index shipped to it as bytes (`Platform::go_offline`).

5. k-d tree — nearest point

- **Problem:** the nearest of 144,090 road junctions to a responder or an incident.

- **How:** median split (`std::nth_element`) on alternating axes; nearest-neighbour search visits the far side of a split only when the split plane is no farther than the best point so far.
- **Implementation:** `index::KdTree<Id>` (`include/safetrail/index/kd_tree.hpp`), used by `graph::RoadGraph::nearest_node()`. A linear-scan oracle (`nearest_node_linear`) uses the same metric so they agree node for node.
- **Cost:** build O(n log n), query O(log n) expected.

6. AVL interval tree — the time dimension

- **Problem:** zones have validity windows; history asks "which were in force at t?"
- **How:** a BST keyed on (start, end, insertion sequence), AVL-balanced, **augmented** with the maximum end time in each subtree. A stabbing query skips every subtree whose maximum end is ≤ t.
- **Implementation:** `ds::IntervalTree<T>` (`include/safetrail/ds/interval_tree.hpp`), index-based nodes with a free list, single-path deletion, a general rebalance used by both insert and delete, and `check_invariants()` (BST order, balance, augmentation, size) run after every operation in its tests.
- **Cost:** insert/delete O(log n); stab O(min(n, (k + 1) log n)).
- **Lab:** each operation's tree, balance factors, heights and rotations; the nodes a stab entered.

7. Persistent quadtree — every past version

- **Problem:** "what were the rules at 14:32?" for every past moment, cheaply.
- **How: path copying** — an update copies only the root-to-leaf path it changes; every other node is shared by reference (`shared_ptr<const Node>`) with the previous version. Validity changes copy nothing: they append one record to the zone's history, and an interval tree over all historical windows answers time queries.
- **Implementation:** `index::VersionedIndex` (`src/index/versioned_index.cpp`).
- **Where:** zone history in the platform; the Investigation page's "zones in force at t".
- **Cost:** O(depth) new nodes per geometry change; a query against the past costs the same as one against the present.
- **Lab:** two consecutive versions with the copied path in amber and shared nodes in grey.

8. Graph (adjacency list), Dijkstra, A*

- **Problem:** route a responder along roads.
- **Representation:** `graph::RoadGraph` — `vector<vector<Edge{to, weight_m}>>`, weights validated non-negative and finite at insertion (Dijkstra's correctness depends on it).
- **Dijkstra:** binary-heap frontier with lazy deletion; deterministic tie-breaking on node id.
- **A*:** f = g + h, h = great-circle distance to the target, which never exceeds a road distance (`heuristic_is_admissible()` checks it), so A* returns a shortest path while settling fewer nodes.
- **Where:** `Platform::recommend` (one Dijkstra from the incident, valid because the graph is undirected), `Platform::route_between` (A*).
- **Lab:** settle order step by step, the heap's contents before each pop, the two searches' work side by side.

9. Binary heap

- **Implementation:** `ds::BinaryHeap<T, Less>` (`include/safetrail/ds/priority_queue.hpp`), array-backed, sift up/down.
- **Where:** Dijkstra and A* frontiers; `alert::TriageQueue`, which orders open alerts by severity, age and isolation.

10. Hungarian assignment

- **Problem:** several incidents, several free responders — minimise total travel.
- **Implementation:** `graph::hungarian` (`src/graph/bipartite_match.cpp`), O(n^3), validated input (ragged rows, NaN, more rows than columns are rejected with a status). Greedy cheapest-pair-first (`dispatch::assign_greedy`) is kept beside it for comparison.
- **Where:** Incidents → "Batch dispatch: greedy vs optimal".

11. Union-find with rollback

- **Implementation:** `ds::RollbackDSU` — union by rank, no path compression (so every change can be undone), an undo stack, `snapshot()` / `rollback_to()`.
- **Where:** `group::CohesionMonitor` (members within the proximity distance are united; more than one component = the group has split) and `alert::Correlator` (alerts close in space and time are united into one incident).

12. Open-addressing hash table

- **Implementation:** `ds::HashMap<K, V, H>` — power-of-two table, Fibonacci hashing, linear probing, tombstones, rehash at 70 % occupancy counting tombstones, grow only when live entries need it.
- **Where:** usernames → ids; API sessions (keyed by SHA-256 of the token); alert escalation records; offline-sync de-duplication by (Lamport counter, device); per-actor audit throttling.

13. Hashed timing wheel

- **Implementation:** `ds::TimerWheel<Id>` — slots of one tick; a deadline goes to slot (expiry tick mod slots) and fires on the first tick at or after it; cancellation removes it from its slot.
- **Where:** `alert::EscalationTracker` (wheel + hash map): an operator alert unacknowledged for 2 minutes escalates one severity level. The platform advances it once per simulated second, so each advance is O(1) plus what fires.

14. Circular buffer

- **Implementation:** `ds::CircularBuffer<T, N>`, N = 64 for GPS pings.
- **Where:** speed and heading estimation, the stationary and signal-lost anomaly detectors.

15. Merkle tree (evidence log)

- **Implementation:** `evidence::MerkleLog` — RFC 6962 hashing (leaf = SHA-256(0x00 ‖ entry), node = SHA-256(0x01 ‖ left ‖ right)), inclusion and consistency proofs, validated binary save/load; SHA-256 itself in `src/evidence/sha256.cpp`.
- **Where:** every platform log entry is a leaf; the access audit log is a second Merkle log. The Investigation page re-verifies proofs in the browser with WebCrypto; `tools/api_e2e.py` re-verifies with Python's hashlib.

16. Lamport clocks and the offline queue

- **Implementation:** `sync::LamportClock`, `sync::OfflineQueue` (append-only, survives restart via validated file format), `sync::Reconciler` (idempotent merge, ordered by (counter, device), de-duplicated in a hash set).
- **Where:** offline phones in the platform; the offline scenario.

Algorithm complexity

n = number of items, k = number of results, V = vertices, E = edges, P = polygon vertex count, U = tracked users, A = alerts. "Guaranteed" means a proven worst-case bound; "expected" depends on the data's distribution; "measured" means a number from `make bench` (../RESULTS.md).

1. Spatial indexes (range query "which boxes meet this box?")

Structure	Build	Query	Insert	Delete	Space	Kind of bound
Brute force	O(n)	Θ(n)	O(1)	O(n)	O(n)	guaranteed
Quadtree	O(n log n) expected	O(log n + k) expected, O(n) worst	O(log n) expected	O(n) find + O(depth)	O(n)	expected
R-tree (insertion, quadratic split)	O(n log n) expected	O(log n + k) expected, O(n) worst	O(log n) amortised	O(n) find + condense	O(n)	expected
R-tree (STR bulk load)	O(n log n) (the sorts)	same	same	same	O(n), fewer nodes	expected
Geohash (sorted Morton keys)	O(n log n)	O(log n + candidates)	O(n) (sorted array)	O(n)	O(n), no pointers	expected
k-d tree (nearest point)	O(n log n)	O(log n) expected, O(n) worst	rebuild	rebuild	O(n)	expected

Why the worst cases are O(n): a quadtree query that overlaps boxes straddling the root's split lines must look at all of them; an R-tree whose envelopes all overlap the query descends every branch. Removal is O(n) in every index here because none keeps an id → node map (an honest limitation, see LIMITATIONS_AND_FUTURE_SCOPE.md).

2. Trees over time

Operation	Bound	Kind
AVL interval tree insert / delete	O(log n), height ≤ 1.44 $\log_2(n + 2)$	guaranteed
Stabbing query (intervals containing t)	O(min(n, (k + 1) log n))	guaranteed
Persistent quadtree update (geometry)	O(depth) new nodes, rest shared	expected depth
Persistent quadtree validity change	O(log n) (interval tree) + 0 new nodes	guaranteed
Query at a past version	same as at the present	—

3. Graphs and assignment

Algorithm	Bound	Notes
Dijkstra (binary heap, lazy deletion)	O((V + E) log V)	each push O(log V); stale pops skipped
A* (admissible heuristic)	O((V + E) log V) worst; settles ≤ Dijkstra in practice	measured: section 11
Hungarian assignment	$O(n^3)$	n = max(responders, incidents)
Greedy cheapest-pair	O(r · i · min(r, i)) as written	no optimality guarantee
Union-find (rank, no compression)	O(log n) per find/union; rollback O(1) per undone union	compression would make rollback impossible
k-d tree snap	O(log V) expected	replaces an O(V) scan

4. Hashing, queues, time

Structure	Bound
Hash table put/get/erase	O(1) expected at load ≤ 0.7; O(n) worst; rehash O(n) amortised O(1)
Binary heap push/pop	O(log n); top O(1)
Timer wheel schedule	O(1); cancel O(slot size); advance O(Δticks + fired + held)

Structure	Bound
Circular buffer	O(1)
Merkle append / prove / verify	O(1) amortised append (root recomputed O(n) on demand); proof and verification O(log n)
Lamport merge	O(m) de-duplication (hash set) + O((n + m) log(n + m)) sort

5. The geofencing pipeline per GPS fix (`fence::Evaluator::evaluate`)

Step	Cost
usability gate	O(1)
spatial prune	O(log n + k) expected
validity check per candidate	O(1) each (window stored on the zone)
exact containment	O(k · P)
hysteresis + transition	O(1) per candidate
prediction	O(k · P)
reconciliation of open states	O(s), s = zones with open state for this user

Platform ingestion adds O(1) validation and duplicate/order checks, an amortised O(1) trail append (O(log t) insert only for late fixes), and alert bookkeeping proportional to **that user's** alerts (`alerts_by_user_`). Before the per-user index existed the bookkeeping scanned every alert ever raised, O(A) per state change, which made a 5,000-user run quadratic; section 19 of the benchmark found it.

Per platform tick: anomaly detection O(U); cohesion O(Σ g^2) over each group's tracked members; escalation O(1 + fired); retention O(U log t) once a minute.

6. Time–space trade-offs made here

Choice	Pays	Gets
Persistent index instead of full copies	pointer overhead, shared_ptr refcounts	O(depth) per change instead of O(n) per version
Validity on the zone (O(1)) *and* in an interval tree	a second copy of the windows	an O(1) hot path and O(log n)-ish history queries
Hash table tombstones	occasional rebuilds	O(1) delete without breaking probe chains
Rollback DSU without path compression	O(log n) find instead of near-O(1)	the ability to undo
STR packing	needs all items up front	fewer, fuller nodes; faster queries
Geohash flat array	O(n) inserts	a pointer-free index that ships to a phone as bytes

7. Why theory and the stopwatch disagree

- **The answer size bounds the speedup.** At 100,000 zones in one district about 100 zones overlap each query; every index must return them, so the speedup over brute force stops growing (section 1). Hold density constant and it keeps growing (section 2).
- **Constants and caches.** At 100 zones a brute-force scan of a contiguous vector is competitive with a tree walk; the tree pays off as n grows.
- **Expected, not guaranteed.** Clustered data can make a quadtree deep.
- **Measurement noise.** Absolute times on a laptop move between sessions (the R-tree's ratio has ranged roughly 100–235× across sessions), which is why ratios are paired and quoted as ranges (../RESULTS.md).

Testing and benchmarks

The project's rule is that a faster structure may never give a different answer, and that every claim has a measurement or a test behind it. The engine methodology is described in depth in ../TESTING.md and ../RESULTS.md; this document summarises it and adds the platform, API and multi-user evidence produced in the final phase.

1. Results at a glance (2026-10-30, Apple M4, Apple clang 17)

Check	Command	Result
Unit, property, differential, golden, platform, API and HTTP tests	`make test`	**51 files, 12,066 checks, 0 failed**
End to end over real HTTP	`make e2e`	**29 checks, 0 failed**
Mutation testing	`make mutation`	**28/28 injected bugs caught** (23 engine + 5 platform)
Full gate	`make validate`	passes: docs, results tables, `-Werror`, headers, tests, determinism, e2e, UBSan, CMake + ctest, Make-vs-CMake byte-identical output
Benchmark correctness gates	`make bench`	all 8 gates pass (scaling, density, end to end, equivalence, containment, interval, evaluator, multi-user)
Browser	manual, recorded in the work log	sign-in, scenario at 20×, incident acknowledge/assign with road route, Lab views, all 220 evidence entries re-verified in the browser, a forged entry rejected

2. How correctness is established

Layer	Method	Where
Spatial indexes	~55,000 randomised queries per index against brute force over seven hostile workloads, with `check_invariants()` after every operation	`tests/index/differential_test.cpp`, `churn_test.cpp`, `equivalence_test.cpp`
Interval tree	against a linear scan; AVL and augmentation invariants after every insert and delete	`tests/ds/interval_tree_test.cpp`
Persistent index	every past version against a full-table replay	`tests/index/persistence_differential_test.cpp`
Geometry	ray casting vs winding number; sweep line vs $O(V^2)$ pairwise; holes, poles, wraparound	`tests/geo/`
Engine end to end	the full simulation under all four indexes emits byte-identical events	`tests/fence/index_independence_test.cpp`
Determinism	same seed, same bytes; Make and CMake builds agree; macOS and Linux dashboards compared in CI	`make determinism`, CI
Crypto	HMAC-SHA256 and PBKDF2-HMAC-SHA256 against RFC 4231 and RFC 7914 vectors	`tests/app/crypto_test.cpp`
Multi-user isolation	two users entering the same zone independently; one leaving while the other stays; one stopping tracking; alerts attributed to the right person	`tests/app/multi_user_test.cpp`
Ingestion boundary	duplicate (identical and by sequence number), out-of-order (recorded, not evaluated, trail stays sorted), GPS jump, inaccurate fix, invalid coordinates, future timestamps, fixes without consent (refused, not stored), missing fixes → SignalLost → resolved on contact	`tests/app/multi_user_test.cpp`
Consent and access	nobody but the user grants consent or starts sharing; responders see a location only while assigned; reads are audited	`multi_user_test`, `tests/app/api_test.cpp`
Evidence	every log entry's inclusion proof verifies; an edited entry does not; no coordinates in evidence	`multi_user_test`, `tools/api_e2e.py`, browser

Layer	Method	Where
Scenarios	each demo tells its story: no serious alert on the normal walk; approach → uncertain → enter → incident → dwell → exit for the entry; a group split with confirmed straggler and one incident; offline entry arrives late and exactly once, the retry is rejected event for event, Lamport stamps preserved, history shows only what was known; identical event log and Merkle root across runs	`tests/app/scenario_test.cpp`
Incidents and dispatch	acknowledge, assign (A* route; A* settles no more than Dijkstra), resolve frees the responder and resolves the alerts; reassignment	`scenario_test`, `api_e2e`
Groups	split and reunite; straggler confirmed only after the separation lasts	`scenario_test`
Zones	self-intersecting polygon rejected; import all-or-nothing; validity changes	`api_test`
HTTP	limits enforced before buffering; chunked, duplicate Content-Length, malformed request lines and headers refused	`tests/net/http_parse_test.cpp`
API security	401 without session, 403 without CSRF header, throttling after 5 failures, idle expiry, logout, role matrix, path traversal (plain and encoded), bounded lab inputs	`api_test`, `api_e2e`

Mutation testing (do the tests catch bugs?)

`tools/mutation_check.py` copies the tree, injects one realistic bug at a time, rebuilds, and requires a named test to fail. The 23 engine mutants (index pruning, R-tree box growth and reinsertion, AVL rotation and augmentation, persistent-index history, evaluator reconciliation, hysteresis, containment, sweep line) were joined by five platform mutants: fixes accepted without consent, out-of-order fixes evaluated, responder access surviving resolution, an operator able to grant consent, and the reconciler admitting a retried batch twice. All 28 are caught.

Defects found during this phase

Defect	How it was found	Fix
`ZoneStore::load_geojson` left a half-loaded store when a later feature was invalid	writing the import API	all-or-nothing parse
Group separation never alerted; `confirm_ms` never read	audit	separation-duration rule in the platform
Stationary alerts re-raised every tick (each fix reset the confirmed anomaly)	scenario test (1,246 alerts)	only a lost signal clears on contact
People resting together flagged as stationary	scenario test	stationary counts only without a group member nearby
Scenario "inside" target fell outside concave/thin polygons	scenario test	interior-point grid search; thin OSM polygons documented
A phone with a slow clock saw no zones in force	scenario test	documented limitation; demo phone clock is fast instead
Quadratic alert bookkeeping (every state change scanned every alert)	benchmark section 19	per-user alert index
Reconciler de-duplication was O(n) per event	audit	hash set of (counter, device)

3. Benchmarks

Protocol (unchanged from the engine): work precomputed; each sample ≥ 20 ms; 11 interleaved rounds with rotated order; medians and IQR; paired speedups; result checksums as correctness gates. Environment recorded in `bench/results/environment.txt` (this run: commit 9d33bb6, Apple M4, battery).

Comparison	Result (this run)	Source
Range query, 100,000 zones in one district (k ≈ 98)	brute 245 µs; quadtree 32.7×, R-tree 224× faster	`index_scaling.csv`
Same at constant density (k ≈ 1)	quadtree 556×, R-tree 1027×	`index_density.csv`

Comparison	Result (this run)	Source
R-tree, STR vs insertion at 100,000 zones	6.3× faster queries, 14,383 vs 21,368 nodes, depth 5 vs 6	`index_build.csv`
Interval tree vs scan (selective windows)	14–57× faster; **slower** than the scan for dense windows at 1,000 intervals (0.61×)	`interval_tree.csv`
Persistent quadtree	11–13× fewer nodes than full copies at 1,000–5,000 versions; past queries cost the same as present	`versioned_index.csv`
A* vs Dijkstra	71–78 % fewer nodes settled on grids of 64–1,024 junctions	`routing.csv`
Hungarian vs greedy dispatch	6.5–15.7 % less total travel; never worse	`dispatch.csv`
Adaptive sampling (8 h trek model)	257 fixes instead of 28,800 at 100 % near-zone recall (energy is a model)	`power.csv`
Hysteresis on realistic drift	16,168 naive transitions → 502 (target 521)	`hysteresis_ab.csv`
Whole evaluator per fix	5.0 µs at 5,038 zones → 5.7 µs at 50,038 at constant density	`evaluator_cost.csv`
Multi-user platform ingestion	2.7–3.8 µs per fix, flat from 10 to 5,000 users (≈ 300,000 fixes/s on one core); ≈ 6.9 KB per tracked user; noisy stream counters exact	`multi_user.csv`

Single-user versus multi-user: with one user the per-fix cost includes cold caches and fixed overhead; from 10 users up it is flat, which is the evidence that users are independent and nothing in the path scales with the number of other users. The three update patterns cost the same within noise; the noisy stream records fewer transitions (5,452 vs 5,581 at 5,000 users) because late fixes are recorded but not evaluated, by design.

Why theory and measurement differ

See ALGORITHM_COMPLEXITY.md §7: the answer size k caps the speedup; small inputs favour the contiguous scan; laptop timings drift between sessions (ratios are quoted as ranges in RESULTS.md).

4. Reproduce

```
make test && make e2e
```

```
make bench
```

```
make validate
```

```
make mutation
```

5. What is not covered

Load across many processes or machines; real GPS traces and real phones; browser-automation tests of the web app (it was verified by hand in the browser, and its API by `api_e2e.py`); fuzzing of the parsers beyond the targeted malformed-input tests; an external security review.

Security and privacy review

SafeTrail handles the location of people, so it is reviewed as a system that could be misused for surveillance, not only as one that could be attacked. This document records the threat model, what is implemented, how each control is tested, and what remains open. It is honest about scale: this is a single-process system reviewed by its developer, not an audited product.

1. Threat model

Asset	Threat	Actor
Live location and movement history	read by someone with no need	curious operator, other tourist, outsider
Accounts	password guessing, session theft, CSRF	outsider, malicious web page
Evidence log	after-the-fact editing to hide or invent an event	insider
Zone configuration	malformed or malicious GeoJSON	administrator mistake, uploaded file
Server	resource exhaustion (huge bodies, slow clients, many connections, heavy lab requests)	outsider
Secrets	credentials committed to the repository	developer mistake

2. Privacy by design

Principle	Implementation	Tested by
Opt-in only	a tourist must give consent **and** open a session; nobody else can do either for them — not an operator, not an administrator (`Platform::set_consent`, `start_sharing`)	`tests/app/multi_user_test.cpp`, `tests/app/api_test.cpp`
No storage without consent	a fix from a user without consent or a session is refused and stored nowhere	`multi_user_test` ("not stored anywhere")
Visible tracking state	"Location sharing is ON/OFF" banner on the tourist's page; "Tracking active" chips for operators	web app
Stop at any time	`stop_sharing` ends the session and discards zone state (no invented exit event)	`multi_user_test`
Minimal data	username, display name, role; no phone number, email, ID document or real name required	`app::User`
Role-based access	admin / operator / responder / tourist checked in the API **and** again in the platform (two gates)	`api_test` role matrix
Need-to-know for responders	a responder sees a tourist's location only while assigned to an open incident affecting that tourist; access ends when it resolves	`api_test`
Audited access	every location read by someone other than the subject (profile, trail, history, live map) goes into an append-only Merkle-backed audit log; repeated reads within a minute are recorded once	`multi_user_test`, `api_test`, Settings page
Retention	movement trails older than the retention period (default 6 h, configurable) are deleted every minute	`Platform::enforce_retention`
Erasure	a tourist or administrator can erase a movement history; positions on **closed** alerts are redacted; positions on open alerts are kept until they close, so a responder on the way can still find the person — a deliberate, documented exception	`multi_user_test`
Evidence without coordinates	evidence-log entries carry pseudonymous ids and zone ids, never coordinates, so erasure never conflicts with an append-only log	`multi_user_test` ("no coordinates")
Honest demonstration data	every scenario person is created with source "scenario", named "SIM · …" and badged SIM; demo responders are named "Demo …" and badged DEMO; production mode creates neither and refuses scenarios	`tests/app/scenario_test.cpp`, `tools/api_e2e.py`

3. Authentication and sessions

Control	Detail
Password storage	PBKDF2-HMAC-SHA256, 16-byte random salt, 120,000 iterations, self-describing record `pbkdf2-sha256$<it>$<salt>$<hash>`; verified against RFC 7914 and RFC 4231 vectors (`tests/app/crypto_test.cpp`). Argon2id would be the production choice; PBKDF2 is what SP 800-132 still permits and is small enough to implement and verify here.
Timing	constant-time comparison; a login for an unknown username still runs a key derivation, so response time does not reveal which usernames exist.
Session tokens	256 bits from `/dev/urandom` (fail closed if unavailable); the server stores only SHA-256(token).
Cookie	`HttpOnly; SameSite=Strict; Path=/; Max-Age=28800`; `Secure` when `secure_cookies` is set behind TLS.
Expiry	30 minutes idle, 8 hours absolute; deactivating an account ends its sessions on the next request; logout deletes the record.
Throttling	five failed logins for a username or a client address lock it for 60 s.
CSRF	every POST/PATCH/DELETE must carry `X-SafeTrail-Request: 1`. A cross-site form cannot set it, and a cross-site script cannot send it without a CORS preflight this server never approves. SameSite=Strict is a second layer.

4. Input validation and parsing

- **HTTP:** request line and header grammar enforced; header block ≤ 16 KB, ≤ 64 headers, body ≤ 4 MB checked from Content-Length *before* buffering; chunked encoding refused (501); duplicate Content-Length refused (request-smuggling shape); 15 s idle timeout; ≤ 256 connections (`tests/net/http_parse_test.cpp`).
- **JSON:** the project's parser bounds nesting depth (64), rejects trailing data, raw control characters and invalid escapes (../../include/safetrail/util/json.hpp).
- **GeoJSON:** every number range-checked before conversion (out-of-range conversion is undefined behaviour in C++); coordinates finite and on the lat/lon domain; polygons validated (self-intersection via the sweep line, holes inside the outer ring); vertex and zone-count caps; **all-or-nothing import** — one bad feature adds nothing.
- **Fixes:** finite coordinates in range, accuracy in (0, 5000] m, timestamp not more than 60 s in the future nor before the session began.
- **Static files:** an allow-list of characters and no `..`; nothing outside `web/app/` (or the read-only `bench/results/` whitelist) can be named, including percent-encoded traversal (`api_test`, `api_e2e`).
- **Lab:** every experiment bounds its inputs (points, grid size, operation count).

5. Browser hardening

`Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self'; frame-ancestors 'none'; base-uri 'none'; form-action 'self'`, plus `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy` denying geolocation, camera and microphone; `Cache-Control: no-store` on every API response. The web app builds every text node with `textContent`, never `innerHTML`, so user-supplied names and notes cannot inject markup. No third-party scripts, fonts or tiles are loaded.

6. Secrets

No credential is in the repository. Passwords come from `SAFETRAIL_<ROLE>_PASSWORD` environment variables; otherwise they are generated at random on first start and stored in `var/demo-credentials.txt` (mode 0600, directory 0700, git-ignored). `make app-reset` deletes it. The existing `.gitignore` already excludes `.env*` (the Vercel CLI token).

7. Deployment posture

The server binds to 127.0.0.1 by default and prints a warning if told to bind elsewhere: it has **no TLS**. Exposing it beyond the machine needs a TLS-terminating reverse proxy and `secure_cookies`. The public GitHub Pages and Vercel deployments serve only the static dashboard (a replay with no accounts or live data), not this server.

8. Dependencies

None at run time beyond the C++ standard library and POSIX sockets. The build needs a C++17 compiler and make; the tools need Python 3's standard library. The supply-chain surface is the compiler toolchain and the GitHub Actions used by CI (pinned to major versions).

9. Open items (known, not hidden)

1. No TLS in process (by design; see §7).
2. All state is in memory: accounts, zones and evidence are lost on restart. Persisting them (and signing published Merkle roots) is future work.
3. PBKDF2 rather than a memory-hard hash.
4. Throttling and sessions are per process; a multi-instance deployment would need a shared store.
5. Rate limiting covers login only; an authenticated client could still issue many heavy lab requests (each is bounded, but there is no per-user quota).
6. The audit log records reads made through the API; anyone with shell access to the server process is outside its scope.
7. This review was made by the developer. No external penetration test has been performed.

Limitations and future scope

Stated plainly, so that nothing on the slides claims more than the code does.

Limitations of the data and the evaluation

1. **Simulated people, real geography.** Hazard zones and roads are real OpenStreetMap data around Shillong, Meghalaya; every tourist is simulated. No field trial has been run and no real GPS trace has been evaluated.
2. **The OSM hazard polygons are thin.** The rivers ("Deep Water") are at most 25–40 m deep and the Sonapani cliff about 23 m. With a 15 m entry margin and 5–18 m GPS accuracy, a person is rarely *confirmed* inside them; the engine correctly reports "uncertain". The entry scenario therefore uses a larger restricted closure registered by the administrator, labelled SIM.
3. **Benchmarks come from one laptop** (Apple M4). Ratios transfer better than absolute times, and even ratios move between sessions — the R-tree's 100,000-zone ratio has ranged roughly 100–235×. They are quoted as ranges.
4. **The battery figure is a model** (energy proportional to fixes taken, 1 fix/s = 10 %/hour), not a measurement on a phone.
5. **No travel-time estimates.** The road file has segment lengths, not speeds or road classes, so routes report distance only.

Limitations of the engine

1. **Removal is O(n)** in every spatial index (no id → node map).
2. **Worst-case queries are O(n)** for the quadtree and R-tree on adversarial data; the bounds are expected, not guaranteed.
3. **Tolerance-based geometry**, not exact predicates (on-edge tolerance ~0.1 mm).
4. **No antimeridian support**: zones crossing ±180° are refused.
5. **A GPS jump inflates the query radius** for about a minute (the speed window), reducing pruning for that person — correct, but slower.
6. **Prediction is a straight-line extrapolation** over at most 5 minutes.

Limitations of the platform

1. **In-memory only.** Accounts, zones, alerts and the evidence log are lost when the server stops. A production system needs a durable store and signed, published Merkle roots.
2. **Single process, single thread.** About 300,000 fixes per second on one core is far beyond the demo, but there is no horizontal scaling, no replication and no failover.
3. **Late fixes are recorded, not evaluated.** Re-evaluating them would mean rewinding the hysteresis state machine; an event-sourced design could replay a user's window instead.
4. **A phone judges zone validity on its own clock.** A phone whose clock is wrong may treat a zone as in force too early or too late while offline; hysteresis timers handed back from a phone whose clock was fast confirm late by up to the skew. Server-side skew correction applies only to the events, after reconnect.
5. **One Dijkstra gives road distance to every responder** only because the road graph is undirected; a directed graph (one-way streets) would need a reverse-graph search.
6. **Group cohesion is $O(g^2)$** per group per pass (all pairs); fine for tour parties, not for crowds. A grid bucket would make it O(g) expected.
7. **No push notifications.** The console polls once a second; tourists have no phone app — the "simulated phone" on the tourist page stands in for one.
8. **Security** — see the open items in SECURITY_AND_PRIVACY.md (no in-process TLS, PBKDF2 not Argon2, no external penetration test).

Future scope

Area	Next step
Field data	evaluate on recorded GPS traces with ground truth; tune hysteresis margins per zone type
Persistence	write-ahead event log replayed on start; durable Merkle log with signed roots and consistency proofs published to responders
Mobile	a phone app that runs the same evaluator offline from the shipped geohash bytes (the platform already simulates exactly this)

Area	Next step
Scale	partition users across worker threads by user id (users are independent), keeping one writer per user; grid bucketing for cohesion
Indexes	id → leaf maps for O(log n) removal; Hilbert-ordered R-tree packing; a centred interval tree for O(log n + k) stabbing
Routing	travel-time weights from road classes; contraction hierarchies for city-scale graphs; multi-objective dispatch (time, unit type)
Jurisdiction	resolve each alert's owning district through the existing `jurisdiction::Hierarchy` (tested, not yet wired in)
Privacy	differential-privacy aggregates for analytics; per-zone consent scopes
Security	TLS, Argon2id, per-user request quotas, external security review